M4010104 / M4020104: PROGRAMMING IN PYTHON · DIGITAL UNIVERSITY KERALA · CAPSTONE PROJECT SPECIFICATION

Project 01: IoT Environmental Sensor Telemetry Curation & Anomaly Detector

Effort Budget: 10–12 Hours | Core Stack: Pure Python, NumPy, Pandas, Matplotlib, Seaborn, Plotly | Max Marks: 25 | Instructor: Dr. Sunil TT

1. Executive Overview & Domain Context

Environmental telemetry systems deployed across smart cities and industrial hubs monitor critical micro-climate parameters (temperature, humidity, CO, LPG, smoke). Due to low-cost hardware limitations, power supply glitches, and high humidity, electrochemical sensors frequently report corrupted telemetry, sudden baseline shifts, and physical spikes. Uncurated data directly distorts environmental compliance audits and municipal air-quality alerts.

Your team will engineer an end-to-end, production-grade telemetry processing pipeline in pure Python. The pipeline ingests raw sensor CSV telemetry without database engines, audits domain bounds, models sensor interactions using NumPy linear algebra, tracks rolling baseline envelopes in Pandas, and visualizes multi-sensor dynamics through static and interactive dashboards answering the 12 core questions below.

Primary Public Dataset & Telemetry/Data Schema

The project utilizes the verified public Environmental Sensor Telemetry (132k Readings) on Kaggle (**`https://www.kaggle.com/datasets/garystafford/environmental-sensor-data-132k`**). The dataset structure captures:

- **Raw Feature Attributes:** **`ts`** (epoch timestamp), **`device`** (MAC address), **`co`** (carbon monoxide ppm), **`humidity`** (% RH), **`light`** (boolean/lux), **`lpg`** (ppm), **`motion`** (boolean), **`smoke`** (ppm), **`temp`** (°C).
- **Engineered / Target Variables:** **`datetime`** (UTC parsed), **`co_zscore`**, **`temp_hum_interaction`**, **`rolling_iqr_spike`**, and multi-sensor correlation matrices.

2. Complete Project Directory & File Architecture

Each team must organize their repository into a standardized, production-grade layout. All application source code must reside inside a dedicated package (**`src/`**), with separate directories for raw/curated data, interactive development notebooks, visual figures, and formal documentation:

```
iot-sensor-telemetry-pipeline/
├── README.md                          # Repository documentation & GitHub Pages source
├── requirements.txt                   # Dependency specification (numpy, pandas, matplotlib, seaborn, p
├── run_pipeline.py                    # Standalone production CLI application entry point
├── data/
│   ├── raw/
│   │   └── air_quality_iot_raw.csv            # Raw uncleaned input dataset (including simulated/inject
│   └── processed/
│       ├── air_quality_iot_clean.csv          # Curated, validated clean CSV dataset
│       └── iot_telemetry_audit.log           # Timestamped audit report detailing filtered/rejected rec
├── src/
│   ├── __init__.py                    # Python package initialization
│   ├── ingestion.py                   # Defensive line-by-line file reader, range auditor & CSV exporte
│   ├── analyzer.py                    # IoTSensorAnalyzer OOP class, NumPy vectorization & Pandas analy
│   └── visualizer.py                  # Matplotlib envelope plots, Seaborn heatmaps & Plotly HTML expor
├── notebooks/
│   └── development_cycle.ipynb        # Development cycle notebook: EDA, trials, tests & unit assertion
└── reports/
    ├── figures/
    │   ├── primary_envelope.png       # Matplotlib confidence envelope & regression overlay
    │   ├── correlation_heatmap.png    # Seaborn annotated feature correlation heatmap
```

```
│   └── distribution_regimes.png   # Seaborn distribution across categorical/domain tiers
├── interactive_dashboard.html     # Standalone interactive Plotly HTML dashboard with controls
└── Project_Report.pdf             # 3-4 page final technical report
```

3. Dual Development Deliverables: Notebook vs. Standalone Program

To mirror industry software engineering standards, each student group must deliver two distinct, complementary implementations of their solution:

Deliverable A: The Development Cycle Notebook (`notebooks/development_cycle.ipynb`)

The development notebook chronicles your team's engineering journey from raw data exploration to validated model prototypes. It is not merely a final script pasted into cells, but an authentic record of the development cycle:

- **Phase 1: Exploratory Data Ingestion & Diagnostics:** Loading raw data, inspecting missing values, testing boundary rules, and logging rejected records.
- **Phase 2: Mathematical Formulation & Feature Verification:** Developing and validating NumPy array calculations against target benchmarks using **`np.allclose()`**.
- **Phase 3: OOP Class Architecture & Unit Tests:** Instantiating the **`IoTSensorAnalyzer`** class, executing automated unit assertions (e.g., verifying that passing invalid parameters raises an explicit **`ValueError`**), and testing edge cases.
- **Phase 4: Statistical Modeling & Regimen Exploration:** Implementing ordinary least-squares regression, rolling-window calculations, and multi-metric Pandas grouping.
- **Phase 5: Visual Prototype Development:** Iteratively designing and refining Matplotlib and Seaborn figures, followed by configuring the interactive Plotly HTML dashboard.

Deliverable B: The Final Standalone Python Program (`run_pipeline.py` & `src/`)

While the notebook documents the exploratory development cycle, your repository must also provide a production-ready, fully executable Python application. The program must run end-to-end directly from the terminal without requiring Jupyter:

- **Command-Line Interface (CLI):** Executed via **`python run_pipeline.py --input data/raw/air_quality_iot_raw.csv --output-dir data/processed --report-dir reports`**.
- **Modular Package Import:** The driver script imports cleanly separated domain logic from **`src.ingestion`**, **`src.analyzer`**, and **`src.visualizer`**.
- **Terminal Feedback & Exit Codes:** The program must print structured progress indicators, log clean/faulty record counts to the console, and return an exit code of **`0`** upon successful pipeline completion.
- **Automated Artifact Generation:** In a single execution pass, the script reads raw inputs, writes **`data/processed/air_quality_iot_clean.csv`**, saves **`data/processed/iot_telemetry_audit.log`**, exports static figures to **`reports/figures/`**, and generates the standalone **`reports/interactive_dashboard.html`**.

4. Technical Investigation Requirements & Research Questions (Q1–Q12)

Your team's implementation across the development notebook and standalone program must rigorously answer the following 12 course-aligned engineering questions:

Course Topic & Script	Q#	Concrete Engineering & Analytical Task	Python Libraries & Methods
File I/O & Exceptions (`src/ingestion.py`)	**Q1**	**Physical Feasibility & Range Bounds Audit:** Using pure Python open() and try-except blocks (handling ValueError, KeyError), parse the raw CSV telemetry stream. Identify and count how many records contain physically impossible values: temp outside [-10, 60] °C, humidity outside [0, 100] %, or negative gas readings (co, lpg, smoke < 0).	open(), csv.DictReader, try-except (ValueError), conditional bounds checking
Data Curation & Export (`src/ingestion.py`)	**Q2**	**Defensive Data Export & Structured Audit Logging:** Export all valid, physically feasible records to air_quality_iot_clean.csv. Concurrently, create a structured text audit log iot_telemetry_audit.log detailing total records processed, valid row count, corrupted record count, and an itemized list of rejected line numbers with specific failure reasons.	Defensive file I/O, context managers (with open), formatted string reporting
OOP Design (`src/ingestion.py`)	**Q3**	**Object-Oriented Telemetry Analyzer Architecture:** Encapsulate analytical operations inside a production-grade class IoTSensorAnalyzer with methods load_data(), get_summary_stats(), and compute_zscores(). Enforce data encapsulation with private attributes (__raw_data) and raise explicit ValueError exceptions if methods are invoked before data is successfully loaded.	OOP class structure, __init__, private attributes, docstrings, custom exception handling
NumPy Arrays (`src/analyzer.py`)	**Q4**	**Vectorized Normalization & Mathematical Properties:** Convert raw telemetry columns into 1D NumPy ndarrays. Implement a vectorized Min-Max scaling function without loops. Compute array shape, byte size (nbytes), memory layout flags, and verify that the scaled values strictly satisfy min=0.0 and max=1.0 using np.allclose().	np.ndarray, np.min(), np.max(), array slicing, np.allclose()
NumPy Masking (`src/analyzer.py`)	**Q5**	**Multivariate Anomaly Isolation via Boolean Masking:** Compute vectorized Z-scores for co, temp, and humidity. Construct compound boolean masks using bitwise operators (&, |, ~) to identify records where |Z| > 3.0 simultaneously across multiple channels. Compare the masked anomaly subset against raw non-vectorized conditional filtering using Python sets to prove exact match.	np.mean(), np.std(), boolean masks (&, |, ~), bitwise indexing, set comparison
NumPy Regression (`src/analyzer.py`)	**Q6**	**Ordinary Least Squares (OLS) Regression from First Principles:** Formulate a multiple linear regression model: co = b0 + b1*temp + b2*humidity. Using pure NumPy matrix operations (np.linalg.lstsq or the normal equation (X^T X)^(-1) X^T y), estimate the parameter vector beta. Compute the coefficient of determination (R^2) and residual sum of squares (RSS) using vector dot products.	np.column_stack(), np.linalg.inv(), @ matrix multiplication, np.dot()
Pandas Series (`src/analyzer.py`)	**Q7**	**Pandas Series Manipulation & Rolling Window Envelope:** Load clean telemetry into a Pandas DataFrame. Convert the epoch timestamp ts into a Pandas DatetimeIndex. Compute a 15-minute rolling median and rolling Interquartile Range (IQR). Identify transient sensor spikes that exceed 2.5 times the local rolling IQR.	pd.to_datetime(), .rolling(window=15), .quantile(), series arithmetic

Course Topic & Script	Q#	Concrete Engineering & Analytical Task	Python Libraries & Methods
Pandas Groupby (`src/analyzer.py`)	**Q8**	**Multi-Index Aggregation & Pivot Table Synthesis:** Discretize temperature into bins (Cold, Moderate, Warm, Hot) and group by device and hour of day. Build a two-dimensional pivot table showing mean and 95th percentile carbon monoxide levels. Add marginal row/column summaries (margins=True) and flatten the resulting MultiIndex columns into clean snake_case strings.	pd.cut(), .groupby(), .pivot_table(margins=True), MultiIndex column flattening
Pandas Reshaping (`src/analyzer.py`)	**Q9**	**Tidy Reshaping & Long-Format Anomaly Ranking:** Reshape wide multi-sensor measurements (co, lpg, smoke) into a tidy long-format DataFrame using pd.melt() with id_vars=['ts', 'device']. Perform a secondary groupby on sensor_type to extract the top 10 highest anomaly spikes for each gas pollutant.	pd.melt(), tidy data principles, .groupby(), .nlargest()
Matplotlib (`src/visualizer.py`)	**Q10**	**Dual-Axis Telemetry Envelope Visualization:** Using Matplotlib, create a publication-quality time-series figure. On the primary y-axis, plot raw carbon monoxide telemetry with an overlaid 15-minute rolling median and shaded rolling IQR envelope (ax.fill_between()). On a secondary twin y-axis (ax.twinx()), plot ambient temperature. Annotate the single highest anomaly spike with an arrow callout (ax.annotate()).	plt.subplots(), ax.twinx(), ax.fill_between(), ax.annotate(), custom legends
Seaborn (`src/visualizer.py`)	**Q11**	**Statistical Distribution & Multi-Sensor Heatmaps:** Using Seaborn, generate a 2-panel figure: (Left) An annotated Pearson correlation heatmap (sns.heatmap(annot=True)) revealing cross-sensor collinearity; (Right) A split violin plot (sns.violinplot()) comparing CO distribution between Day vs. Night across distinct device MAC addresses.	sns.heatmap(), sns.violinplot(), sns.despine(), palette styling
Plotly HTML (`src/visualizer.py`)	**Q12**	**Interactive Operational Telemetry Dashboard:** Engineer a self-contained, standalone HTML dashboard (interactive_dashboard.html) using Plotly Express and Graph Objects. Include an interactive time-series chart with a range slider, dynamic hover tooltips displaying all sensor readings simultaneously, and visual anomaly threshold lines. Ensure the file renders completely offline without external server dependencies.	plotly.graph_objects.Figure, add_trace(), rangeslider, fig.write_html()

5. Collaborative Git Workflow, GitHub Pages & README.md Specification

Best Practices: Collaborative Git Workflow & GitHub Pages Deployment from Day 1

From the very first day, the student team must adopt professional version control and documentation practices. On Day 1, one designated team member initializes the central public GitHub repository, establishes a clear branch protection rule preventing direct pushes to `main`, adds all teammates as registered **Contributors** (with write access), and grants read access to the Course Instructor (`Dr. Sunil TT`) and teaching assistants. Immediately thereafter, the team must activate **GitHub Pages** (via *Settings → Pages → Build and deployment: Deploy from a branch: main /docs or root*) so that project documentation, interactive architecture diagrams, and the live Plotly dashboard (`reports/interactive_dashboard.html`) are publicly accessible and continuously deployed throughout the development lifecycle. All feature development must follow a strict branching model (e.g., `feature/ingestion`, `feature/analysis`, `feature/visualization`): individual team members must write small, atomic, well-commented commits from their own personal GitHub accounts and merge them into `main` exclusively through formal **Pull Requests (PRs)** accompanied by peer review. Dumping entire codebases in a single, monolithic end-of-semester

commit is strictly prohibited and will result in zero marks for individual Git contribution; maintaining an unbroken, transparent Git commit and PR audit trail is essential for passing the code ownership defense during the live viva-voce examination.

Repository README.md Specification Template

The root **`README.md`** serves as the primary technical landing page and must strictly conform to the following structural format:

```
# Project 01: IoT Environmental Sensor Telemetry Curation & Anomaly Detector
[![Course](https://img.shields.io/badge/Course-Programming%20in%20Python-blue.svg)](https://duk.ac.in)
[![Python](https://img.shields.io/badge/Python-3.11%2B-brightgreen.svg)](https://python.org)
[![License](https://img.shields.io/badge/License-MIT-yellow.svg)](LICENSE)

## 1. Project Overview & Domain Context
- **Institution:** Digital University Kerala (MTech AI / MSc AI)
- **Course:** M4010104 / M4020104 – Programming in Python | **Instructor:** Dr. Sunil TT
- **Dataset:** Environmental Sensor Telemetry (132k Readings) on Kaggle (https://www.kaggle.com/datasets

## 2. System Architecture & Pipeline Design
[Provide an ASCII or Mermaid architecture flow diagram showing raw ingestion -> validation -> analyzer -

## 3. Installation & Environment Setup
```bash
git clone https://github.com/<your-team>/iot-sensor-telemetry-pipeline.git
cd iot-sensor-telemetry-pipeline
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
pip install -r requirements.txt
```

4. Execution Instructions
A. Running the Standalone Application
```bash
python run_pipeline.py --input data/raw/air_quality_iot_raw.csv --output-dir data/processed --report-dir
```
B. Running the Development Notebook
```bash
jupyter notebook notebooks/development_cycle.ipynb
```

5. Key Analytical & Statistical Findings (Q1-Q12 Summary)
[Provide a concise markdown summary table detailing raw vs. clean counts, regression coefficients, anoma

6. Interactive Telemetry Dashboard
Explore the live interactive Plotly visualization hosted on GitHub Pages:
🔗 [Live Project Dashboard](https://<your-team>.github.io/iot-sensor-telemetry-pipeline/interactive_dash

7. Contributor Git Commit & Pull Request Log Table
Contributor Name	GitHub Handle	Assigned Module / Tasks	Verified Commit Hash Range	Merged Pull
[Student Full Name]	`@github_handle`	Modular Architecture (`src/ingestion.py`, `src/analyzer.py`,
```

6. Evaluation Rubric & Mandatory Academic Integrity Governance

Projects are evaluated on a 25-mark scale based on code quality, mathematical correctness, visualization depth, git collaboration discipline, and live technical defense:

Assessment Criterion	Basis of Evaluation & Deliverable Verification	Max Marks
1. Data Curation & File I/O Pipeline (Q1–Q2)	Robust line-by-line file reading with defensive **`try-except`** handling, physical/domain bounds validation, clean CSV output, and structured **`iot_telemetry_audit.log`** reporting.	**5 Marks**

Assessment Criterion	Basis of Evaluation & Deliverable Verification	Max Marks
2. OOP, NumPy & Pandas Analytics (Q3–Q9)	Modular **`IoTSensorAnalyzer`** class architecture, vectorized NumPy feature re-derivations (verified via **`np.allclose`**), regression modeling, and Pandas rolling/groupby/melt operations.	**6 Marks**
3. Multi-Library Visualizations (Q10–Q12)	Publication-quality Matplotlib confidence envelope/trend plot, Seaborn correlation heatmap and distribution violin plot, and standalone interactive Plotly HTML dashboard.	**5 Marks**
4. Software Packaging, GitHub & Documentation	Clean directory layout, comprehensive **`README.md`**, working GitHub Pages site, **`development_cycle.ipynb`** notebook, standalone **`run_pipeline.py`**, and balanced individual Git commits/PRs.	**4 Marks**
5. Live Presentation & Code Viva-Voce Defense	Technical walkthrough and individual live viva defense where each student must explain or modify lines of code in real time without external assistance.	**5 Marks**

> **STRICT ACADEMIC INTEGRITY & ZERO-TOLERANCE AI POLICY (0 MARKS PENALTY):** All submitted Python source code (**`.py`**, **`.ipynb`**) and written documentation will be screened through an automated **AI-Code & Plagiarism Detection Suite**. If AI-generated code or synthetic documentation is detected at any checkpoint or final submission, **0 Marks will be awarded immediately** for the entire project. Each student must prove authentic individual code ownership during the live viva-voce examination — *Course Instructor: Dr. Sunil TT*.

7. Milestone Schedule, Submission Checklist & Viva Defense Prompts

To ensure systematic progress across the semester, teams must adhere to three project checkpoints and prepare for on-the-spot live code modification challenges during the viva-voce examination:

Milestone Stage	Scheduled Window	Required Deliverables & Verification Milestones
Checkpoint 1 (Data Ingestion)	Week 11 Lab Session	Repository initialized with contributors and GitHub Pages active. **`src/ingestion.py`** operational with line-by-line filtering, **`air_quality_iot_clean.csv`** exported, and **`iot_telemetry_audit.log`** generated.
Checkpoint 2 (Analytics & OOP)	Week 13 Lab Session	**`src/analyzer.py`** complete with **`IoTSensorAnalyzer`** class, NumPy feature checks (Q4), outlier masks (Q5), regression modeling (Q6), and exploratory notebook sections.
Final Submission & Viva	Week 15 Project Review	Full package operational: standalone **`run_pipeline.py`**, polished **`development_cycle.ipynb`**, interactive **`interactive_dashboard.html`** on GitHub Pages, 3–4 page PDF report, and live code defense.

Representative Live Viva-Voce Code Modification Challenge

During the individual code viva-voce, the instructor will ask students to execute an immediate 2-minute code modification on their working repository to demonstrate genuine authorship:

- **Challenge Prompt:** *"In src/ingestion.py, modify the temperature physical validity threshold from [-10, 60] °C to [0, 50] °C. Re-run your pipeline and report the exact number of newly flagged records in iot_telemetry_audit.log."*